

TDS200 · Kryssplattform

Session 5

Backend Foundations: Firebase & Firestore

Firebase · Firestore · Security Rules · Setup

Cross-Platform Mobile Development · Lecturer: Yuan Lin

What you'll learn today

- Create a Firebase project and register an app
- Firestore data model: collections, documents, NoSQL
- Read and write your posts in Firebase FireStore



From device to cloud

Posts move out of AsyncStorage and into a shared cloud database.



Project step

Read the feed from Firestore and write new posts to it.

How a 2-hour session runs

0:00-0:05	Recap & objectives Where we are in the project; today's goals
0:05-0:45	Firestore setup & Firestore data model (theory) Firestore Setup, Firestore collections, documents
0:45-1:00	Break —
1:05-1:40	Read & write posts Firestore CRUD from the app
1:40-1:45	Assignment brief & Q&A This week's deliverable, then questions

Create a Firebase project

Step 1 - Create a Firebase project

- Go to Firebase console <https://console.firebase.google.com> (you need to login using your google account)
- Create a Firebase project.
- You can enable AI assistant (Gemini) and Google Analytics.



Create a new Firebase project

Integrate Firebase products to supercharge your app

× Create a project

Let's start with a name for
your project[?]

Project name

my-test-project

my-test-project-d2959

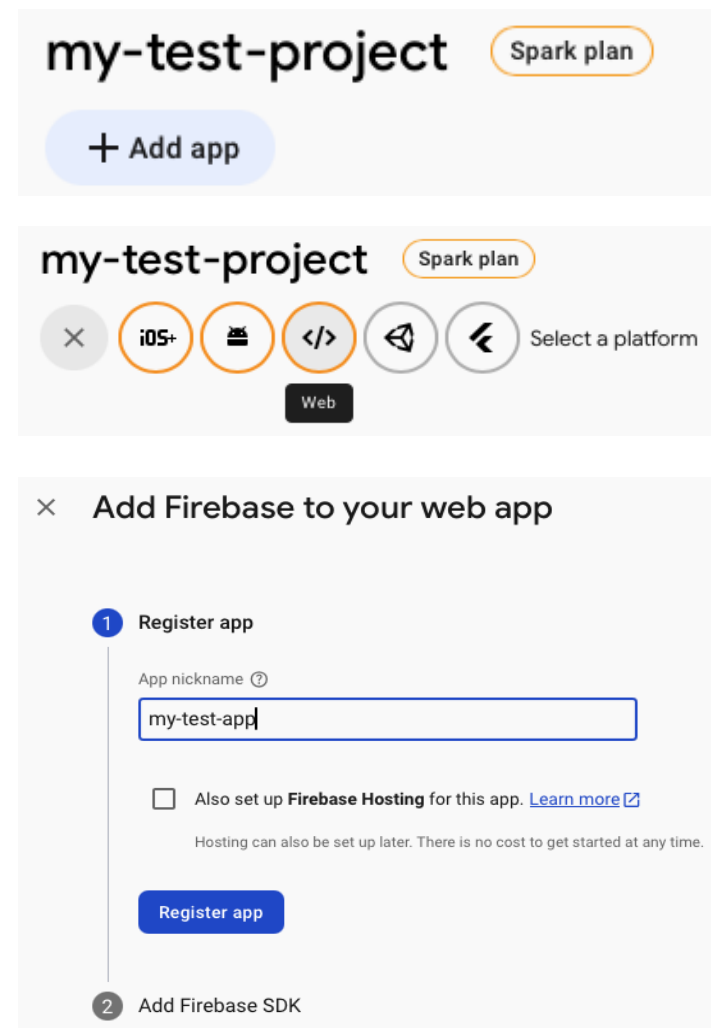
Create a Firebase project

Step 2 - Register an app

- A Firebase project is the backend container for all your services (Auth, Firestore, Storage, etc)
- When adding an app, Firebase will generate configuration keys, such as **apiKey**, **authDomain**, **projectId**, **storageBucket**., required by the selected SDK and platform.
- Multiple client apps can use the same Firebase project and share its backend services.
- Your client app would need these configuration keys to connect to Firebase.
- A Firebase app registration creates a Firebase identification and configuration for an existing client app. A Firebase app is NOT your client app!

You will get platform-specific setup by registering Firebase app.

- For Web and simulators, you get a JavaScript config object to use in your code.
- For Android, you download google-service.json
- For iOS, you download GoogleService-Info.plist.



Create a Firebase project

Step 3 – Add a Web app

- It generates a configuration object for you to use in your client app.
- Then it generates a configuration object for you to use in your client app.

✓ Register app

2 Add Firebase SDK

☒ Use npm ☐ Use a <script> tag

If you're already using [NPM](#) and a module bundler such as [webpack](#) or [Rollup](#), you can run the following command to install the latest SDK ([learn more](#)):

```
$ npm install firebase
```

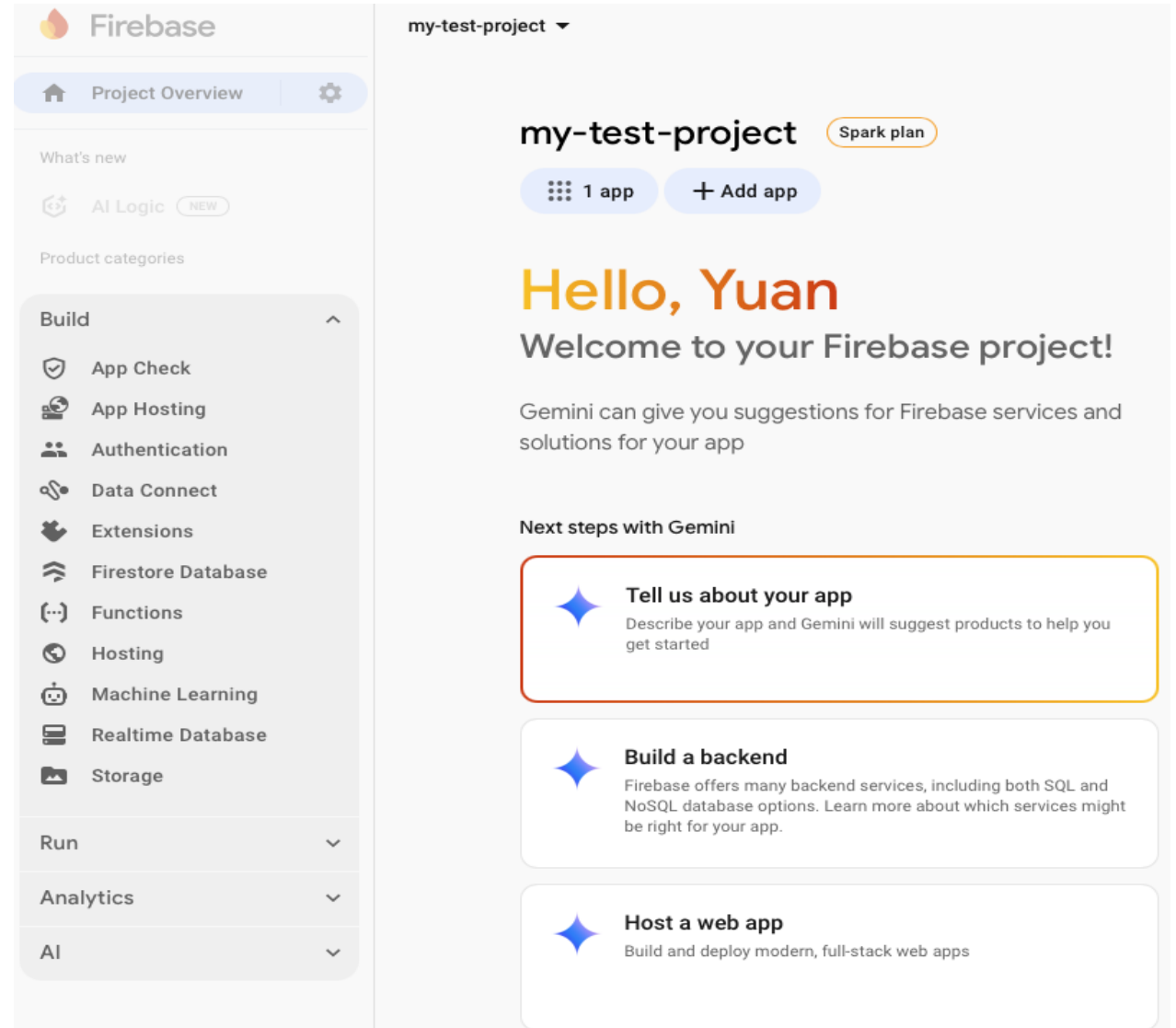
Then, initialise Firebase and begin using the SDKs for the products that you'd like to use.

```
// Import the functions you need from the SDKs you need
import { initializeApp } from "firebase/app";
import { getAnalytics } from "firebase/analytics";
// Your web app's Firebase configuration
// For Firebase JS SDK v7.20.0 and later, measurementId is optional
const firebaseConfig = {
  apiKey: "xxxxxx",
  authDomain: "my-test-project-xxx.firebaseio.com",
  projectId: "my-test-project-xxx",
  storageBucket: "my-test-project-xxx.firebaseio.com",
  messagingSenderId: "xxxx",
  appId: "xxxx",
  measurementId: "xxx"
};
// Initialize Firebase
const app = initializeApp(firebaseConfig);
const analytics = getAnalytics(app);
```

Create a Firebase project

Step 4 - Go to Firebase Console

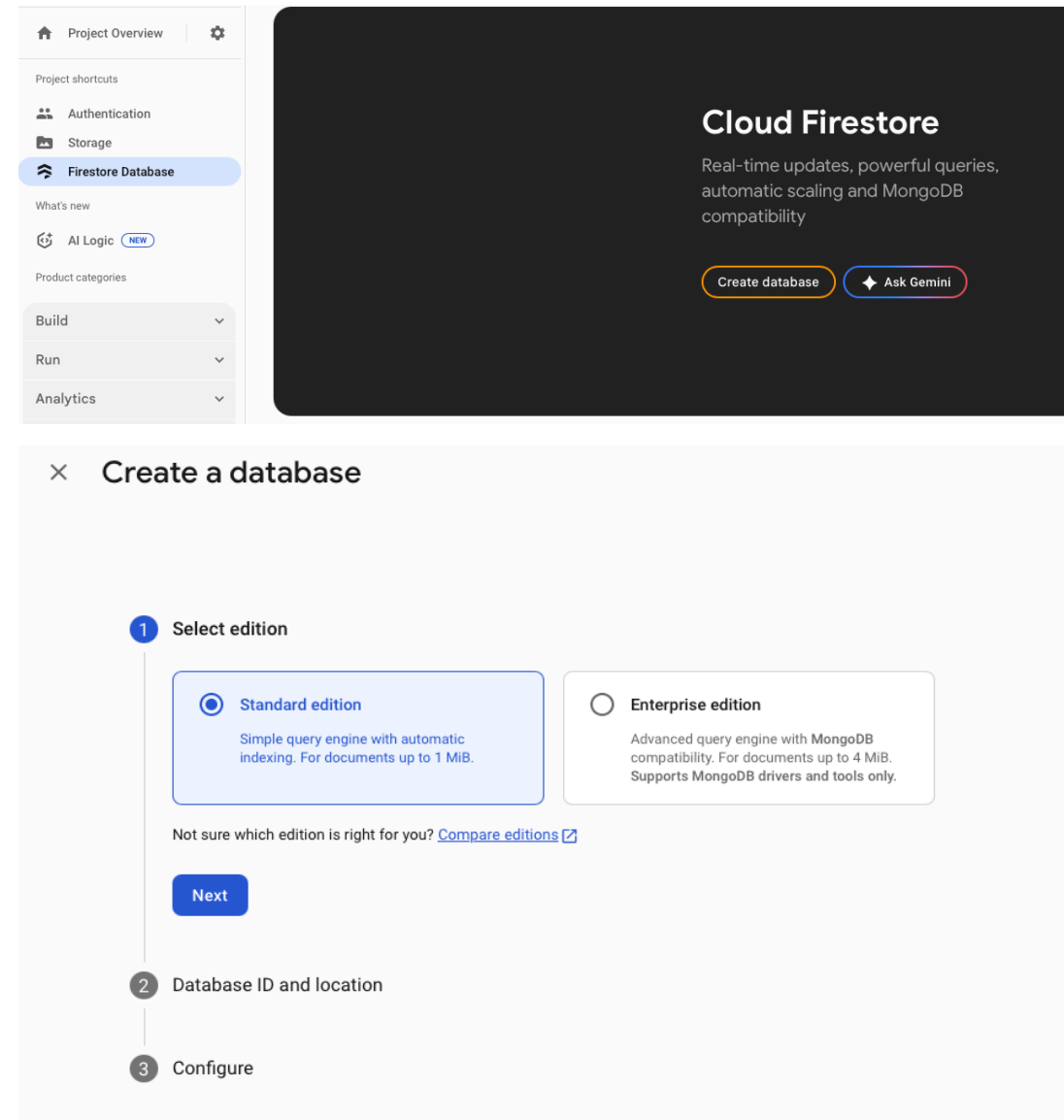
- You can start configuring authentication, Firestore database, and Storage.
- Note that you can always go to Project overview/project settings to find the configuration object.
- From Firebase Console, you can also delete your app or remove the project completely.



Create a Firebase project

Step 5 - Create a Firestore database

- Add Firestore database to your Firebase project.



Create a project - Step by step guidance

Step 5 - Create a Firestore database

- Add Firestore database to your Firebase project. Remember to choose test mode.

×

Create a database

1

Select edition

2

Database ID and location

3

Configure

Database ID

(default)

Location

eur3 (Europe)

ⓘ

Your location setting is where your Cloud Firestore data will be stored

!

After you set this location, you cannot change it later.

Cancel

Next

×

Create a database

1

Select edition

2

Database ID and location

3

Configure

After you define your data structure, you will need to write rules to secure your data. [Learn more](#)

☒

Start in **Production mode**

Your data is private by default. Client read/write access will only be granted as specified by your security rules.

☐

Start in **test mode**

Your data is open by default to enable quick setup. However, you must update your security rules within 30 days to enable long-term client read/write access.

```

rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if false;
    }
  }
}

```

ⓘ

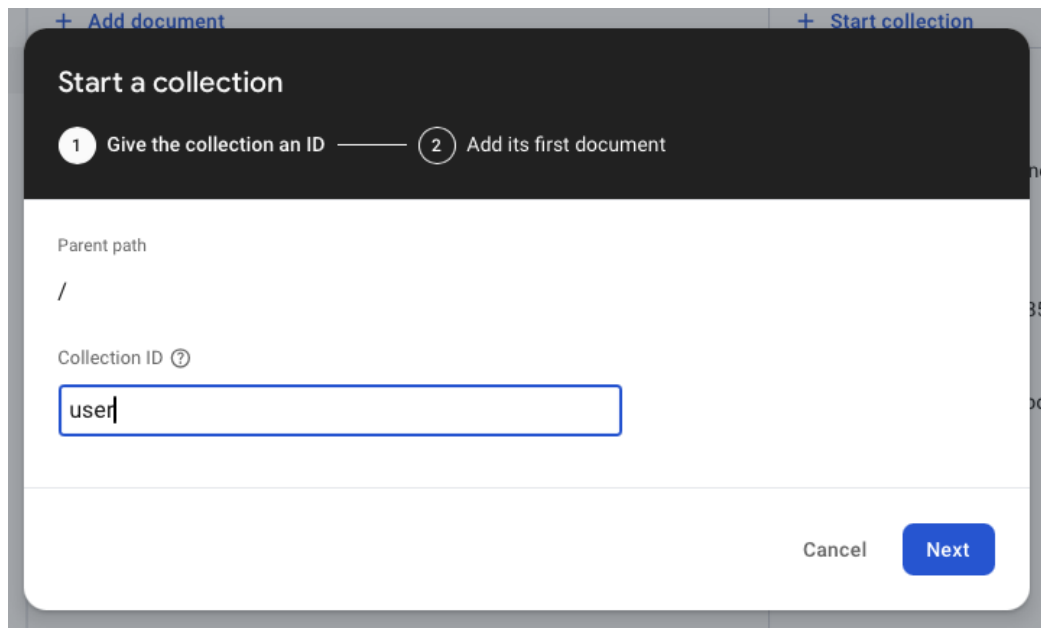
All third-party reads and writes will be denied

Create

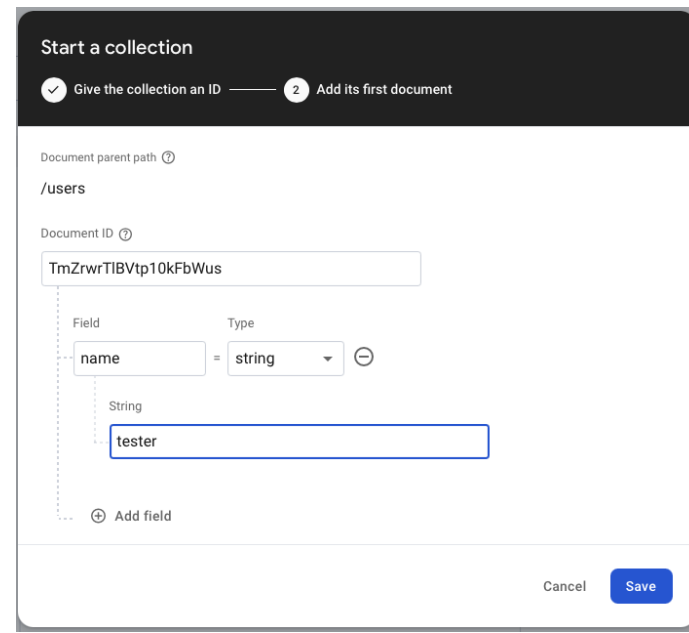
Create a project - Step by step guidance

Step 5 - Create a Firestore database

- Add a collection. You can add collection manually from console.
- Collections are similar to SQL tables and they store related documents.
- Firebase databases are NoSQL-databases, and there is no schema. Documents can have different fields even within the same collection.
- We usually control the structure in the application code, i.e., through TypeScript. Firestore Security rules can also validate fields.



The screenshot shows the 'Start a collection' dialog in the Firebase console. The dialog has a dark header with the title 'Start a collection' and two steps: '1 Give the collection an ID' and '2 Add its first document'. The 'Parent path' is set to '/'. The 'Collection ID' field contains the text 'user'. At the bottom, there are 'Cancel' and 'Next' buttons.

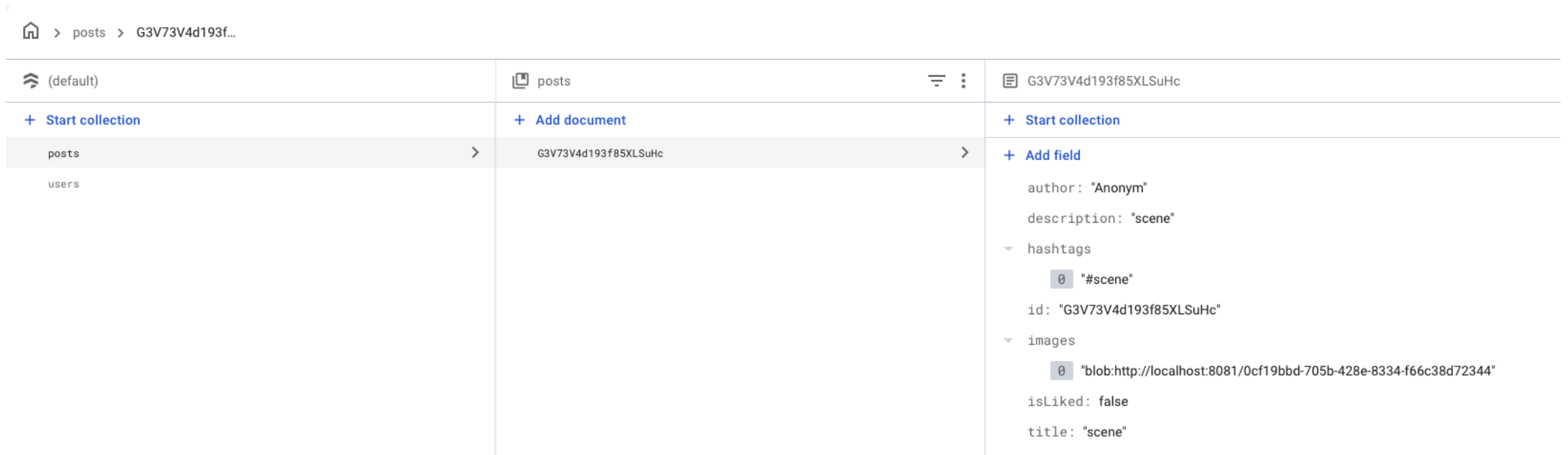


The screenshot shows the 'Add its first document' step of the 'Start a collection' dialog. The 'Document parent path' is '/users'. The 'Document ID' field contains the text 'TmZrwrTIBVtp10kFbWus'. Below this, there is a table with two columns: 'Field' and 'Type'. The first row has 'name' in the 'Field' column and 'string' in the 'Type' column. Below the table, there is a 'String' field containing the text 'tester'. At the bottom, there are 'Cancel' and 'Save' buttons.

Create a project - Step by step guidance

Step 5 - Create a Firestore database

- Back to the Firebase console, you will see a new data collection with first document created.
- You do not have to create a data collection from Firebase console. It can be created programmatically from your application code.



Create a project - Step by step guidance

Step 6 - Copy the configuration object and setup your Expo app.

- Install Firebase SDK.
 - npx expo install firebase
- In the root folder of your app, create two files:
 - ✓ firebaseConfig.ts – contains the Firebase initialization code and exports the Firebase services used by the application
 - ✓ .env – stores sensitive configuration values such as Firebase API keys and project identifiers, keeping them separate from the source code.

```
import { initializeApp } from "firebase/app";
import { getFirestore } from "firebase/firestore";

// Firebase reads these values from the .env file.
// The EXPO_PUBLIC_ prefix makes them available in client-side code.
// Never paste real keys here, always use environment variables.
const firebaseConfig = {
  apiKey: process.env.EXPO_PUBLIC_FIREBASE_API_KEY,
  authDomain: process.env.EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN,
  projectId: process.env.EXPO_PUBLIC_FIREBASE_PROJECT_ID,
  storageBucket: process.env.EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: process.env.EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID,
  appId: process.env.EXPO_PUBLIC_FIREBASE_APP_ID,
};

const app = initializeApp(firebaseConfig);

// db is the Firestore database instance we import in every API function.
export const db = getFirestore(app);
```

Create a project - Step by step guidance

Step 6 - Copy the configuration object and setup your Expo app.

- Usually, you will create .env and .env.example
- .env file contains the actual environmental variables by your application and should never be committed to GitHub.
- .env.example contains the same variables names but with placeholder values. The purpose to have .env.example is to document which environment variables are required. It also helps other developers to set up the project.
- The picture on the right shows .env.example. It is safe to submit to GitHub because it does not contain secrets.

```
# Copy this file to .env and fill in your Firebase project values.  
# Get these from: Firebase Console → Project Settings → Your Apps → SDK setup  
  
EXPO_PUBLIC_FIREBASE_API_KEY=your-api-key  
EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN=your-project-id.firebaseio.com  
EXPO_PUBLIC_FIREBASE_PROJECT_ID=your-project-id  
EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET=your-project-id.appspot.com  
EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=your-sender-id  
EXPO_PUBLIC_FIREBASE_APP_ID=your-app-id
```

CRUD Operations in Cloud Firestore

- CRUD represents the four basic database operations:
 - **Create** — add new data
 - **Read** — retrieve existing data
 - **Update** — modify existing data
 - **Delete** — remove data
- All CRUD operations use the same Firestore database instance
`import { db } from './firebaseConfig'`
- Firestore Security Rules should determine which users are allowed to create, read, update or delete data.

Security

- Firebase Security Rules control who can read, create, update, and delete data in Firestore.
- Even if users access Firebase directly outside your app, the database still checks the rules before allowing the operation.
- For the collection we just made, we specify below rules. This rule means: Anyone can read, create, update and delete posts - including unauthenticated users
- Be careful about giving unauthenticated users write access - can lead to abuse. We will change the security rules in later sessions for better protection.

The screenshot displays the Firebase console interface. On the left, a sidebar contains navigation links: Settings, What's new (with a Phone Verification update), Project shortcuts (Firestore, Storage, Authentication), and Product categories (Databases and storage). The main content area is titled 'Data' and has a 'Rules' tab selected. Below the tabs, a timeline shows rule updates: 'Today • 2:18 pm' (highlighted with a star), 'Jun 8, 2026 • 7:35 pm', 'Jun 6, 2026 • 5:28 pm', and 'Jun 6, 2026 • 2:57 pm'. The right side of the 'Rules' tab shows the rule code editor with the following text:

```

1 rules_version = '2';
2 service cloud.firestore {
3   match /databases/{database}/documents {
4     match /posts/{postId} {
5       allow read, write: if true;
6     }
7   }
8 }

```

Security

- If user authentication is required, Uncaught error will throw claiming insufficient permissions.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    match /posts/{postId} {
      allow read, write: if request.auth != null;
    }

    match /comments/{commentId} {
      allow read, write: if request.auth != null;
    }

  }
}
```

```
> Press ? | show all commands
iOS Bundled 2179ms node_modules/expo-router/entry.js (1161 modules)
ERROR [Error: Uncaught (in promise, id: 0) FirebaseError: Missing or insufficient permissions.] Uncaught (in promise
, id: 0) FirebaseError: Missing or insufficient permissions.
```

Read & write posts

read all posts

```
import { collection, getDocs }
  from 'firebase/firestore';
import { db } from './firebaseConfig';

const snap = await getDocs(collection(db, 'posts'));
const posts = snap.docs.map((d) =>
  ({ id: d.id, ...d.data() }));
```

update a post

```
import { collection, updateDoc }
  from 'firebase/firestore';
import { db } from './firebaseConfig';

const snap = await updateDoc(doc(db, 'posts', postId),
  {title: 'Updated title',
  description: 'Updated description'
  });
```

create a post

```
import { addDoc, collection, serverTimestamp }
  from 'firebase/firestore';
import { db } from './firebaseConfig';
await addDoc(collection(db, 'posts'), {
  title,
  images,
  createdAt: serverTimestamp(),
});
```

delete a post

```
import { doc, deleteDoc }
  from 'firebase/firestore';
import { db } from './firebaseConfig';

await deleteDoc(doc(db, 'posts', postId));
```

Firestore Methods

Create

- `addDoc(collection, data)` – add a document with auto-generated ID
- `setDoc(docRef, data)` – create a document using a specified ID or replace an existing document.
- `setDoc(docRef, data, {merge: true})` – create or update a document without replacing its other fields.

Read

- `getDoc(docRef)` – fetch a single document
- `getDocs(collectionRef)` – fetch all documents from a collection
- `onSnapshot(reference, callback)` – listen for realtime changes to a document, collection or query.
- `doc(db, “posts”, postId)` – create a reference to a specific document
- `collection(db, “posts”)` – create a reference to a collection.

Firestore Methods

Update

- `updateDoc(docRef, {field: value})` – update specific fields in a document
- `arrayUnion(value)` – add a value to an array field and prevent duplicates
- `arrayRemove(value)` – remove a value from an array field

Delete

- `deleteDoc(docRef)` – delete a document

Query

- `query(collectionRef, constraints)` – Build a Firestore query.
- `where(field, operator, value)` – Filter documents by a condition.
- `orderBy(field, direction)` – Sort query results.
- `limit(number)` – Limit the number of returned documents.

APIs to Firebase

- We have created a new api folder, with postApi.ts taking care of all communications with Firebase Firestore.

```
import {
  collection,
  addDoc,
  getDocs,
  getDoc,
  doc,
  updateDoc,
  deleteDoc,
  serverTimestamp,
  query,
  orderBy,
} from "firebase/firestore";
import { db } from "@/firebaseConfig";
import { PostData } from "@/utils/postData";

const POSTS_COLLECTION = "posts";

// NewPost is PostData without the id field.
// Firestore generates the id automatically when we call addDoc.
export type NewPost = {
  title: string;
  description: string;
  hashtags: string;
  author: string;
};
```

```
// CREATE
export async function createPost(data: NewPost): Promise<void> {
  await addDoc(collection(db, POSTS_COLLECTION), {
    ...data,
    createdAt: serverTimestamp(),
  });
}

// READ ALL – returns all posts sorted by newest first
export async function getAllPostsSorted(): Promise<PostData[]> {
  const q = query(
    collection(db, POSTS_COLLECTION),
    orderBy("createdAt", "desc")
  );
  const snapshot = await getDocs(q);
  return snapshot.docs.map((d) => ({ id: d.id, ...d.data() } as PostData));
}

export async function getAllPosts(): Promise<PostData[]> {
  const snapshot = await getDocs(
    collection(db, POSTS_COLLECTION)
  );
  return snapshot.docs.map((d) => ({ id: d.id, ...d.data() } as PostData));
}

// READ ONE – returns a single post by its Firestore id, or null if not found
export async function getPostById(id: string): Promise<PostData | null> {
  const snap = await getDoc(doc(db, POSTS_COLLECTION, id));
  if (!snap.exists()) return null;
  return { id: snap.id, ...snap.data() } as PostData;
}

// UPDATE – only the fields you pass are changed; everything else stays the same
export async function updatePost(id: string, data: Partial<NewPost>): Promise<void> {
  await updateDoc(doc(db, POSTS_COLLECTION, id), data);
}

// DELETE
export async function deletePost(id: string): Promise<void> {
  await deleteDoc(doc(db, POSTS_COLLECTION, id));
}
```

ARCHITECTURE

The api/ folder

- **Why do we need a separate API folder?**
 - We keep all Firestore operations inside an API folder so that screens focus only on the user interface and user interactions.
- **Now each file has a single responsibility.**
 - Screen: UI, user interaction, state management
 - API: Firestore/database operations
 - Firebase Config: Database connection and initialization

Rules: if you see *collection*, *doc*, *updateDoc* in a screen file, move it to api/.

FILES

```
api/  
  postApi.ts      // CRUD to posts  
  commentApi.ts   // CRUD to comments
```

Load Posts from Firebase

- We now load posts from Firebase instead of from AsyncStorage.

Index.tsx

```
import { getAllPosts, createPost } from "@api/postApi";

useFocusEffect(
  useCallback(() => {
    loadPosts();
    loadAuthor();
  }, [])
);

async function loadPosts() {
  const fetched = await getAllPosts();
  setPosts(fetched);
  setLoading(false);
}
```

postDetails/[id].tsx

```
import { getPostById, updatePost, deletePost } from "@api/postApi";

useEffect(() => {
  loadPost();
}, []);

async function loadPost() {
  setLoading(true);
  const data = await getPostById(id);
  setPost(data);
  setLoading(false);
}
```

Load My Posts from Firebase

- We need two different functions in postApi.ts.
- getAllPosts() returns all posts, and sorted by order.
- getAllMyPosts() returns posts created by the signed in user.
- Think twice: Can we add orderBy to the same Firestore query?
- The answer is yes, but Firestore would require you to create a composite index for author and createdAt. The error message normally provides a link for creating it automatically.
- Firebase will provide a link to create a composite index from the error message. Just follow the link and wait until the index from building to enabled.

```
export async function getAllMyPosts(
  authorName: string,
): Promise<PostData[]> {
  if (!authorName?.trim()) {
    throw new Error("Author name is required.");
  }

  try {
    const q = query(
      collection(db, POSTS_COLLECTION),
      where("author", "==", authorName.trim())
    );

    const snapshot = await getDocs(q);
    return snapshot.docs.map((d) => ({ id: d.id, ...d.data() } as PostData));
  } catch (error) {
    console.error("Failed to fetch the author's posts:", error);
    throw new Error("Could not load the author's posts.");
  }
}
```

```
export async function getAllPosts(
  sortOrder: "asc" | "desc" = "desc"
): Promise<PostData[]> {
  const q = query(
    collection(db, POSTS_COLLECTION),
    orderBy("createdAt", sortOrder)
  );

  const snapshot = await getDocs(q);
  return snapshot.docs.map((d) => ({ id: d.id, ...d.data() } as PostData));
}
```

Load My Posts from Firebase

- Index.tsx shall be adjusted accordingly.
- If no user is signed in, it will load all posts.
- If an user is signed in, it will only load the corresponding posts.
- If there are no posts created by the signed in user, error message will show.

```

async function loadPosts(
  order: "asc" | "desc" = sortOrder,
  author: string = authorName
) {
  setLoading(true);
  try {
    if (!author?.trim()) {
      const fetched = await getAllPosts(order);
      setPosts(fetched);
      return;
    }
    const fetched = await getAllMyPosts(author.trim(), order);
    setPosts(fetched);
  } catch (error) {
    console.error("Failed to load posts:", error);
    Toast.show({
      type: "error",
      text1: "Could not load posts",
      text2:
        error instanceof Error
          ? error.message
          : "Please try again.",
    });
  } finally {
    setLoading(false);
  }
}

```

Reuse PostForm for Edit

- We allow Edit and Delete mode inside postDetails page.
- The dependency array of useEffect includes visible. It runs this effect (refill the data) when the modal opens/closes.

postDetails/[id].tsx

```
{/* Reuse PostForm in edit mode — pre-filled with current post data */}
<PostForm
  visible={editVisible}
  authorName={post.author}
  initialData={post}
  onClose={() => setEditVisible(false)}
  onSubmit={handleEdit}
/>
```

PostForm.tsx

```
useEffect(() => {
  if (visible) {
    if (initialData) {
      setTitle(initialData.title);
      setDescription(initialData.description);
      setChips(parseHashtags(initialData.hashtags));
    } else {
      resetForm();
    }
  }
}, [visible]);
```

Reuse PostForm for Edit

- After you finish Editing, `router.back()` will send you back to previous screen.

PostForm.tsx

```
async function handleEdit(updated: PostData) {
  await updatePost(id, {
    title: updated.title,
    description: updated.description,
    hashtags: updated.hashtags,
  });
  // Update local state so the screen reflects the change immediately.
  setPost((prev) => (prev ? { ...prev, ...updated } : prev));
  setEditVisible(false);
  //router.back();
}
```

ASSIGNMENT 5

Posts in the cloud



YOUR TASKS

- Task1: Connect your own Firebase Project.
 - Create a Firebase project and connect it to your app.
 - Create a 'posts' collection in Firestore. Try create, delete, update, read operations.
- Task2: The session project already allows users to delete a post from the post detail screen. Your task is to add a Delete button directly to each post card on the Home Screen. When the button is pressed, delete the selected post from Firestore.
- Task3: add a Category field to the post form and make sure it flows all the way from the UI to Firestore.

DELIVERABLE

A working Firebase-connected app using your own Firebase project. The Home Screen must show posts from Firestore, allow deleting posts directly from each post card, and include a Category field.

EST. EFFORT

≈ 5-7 hours